

articles or technical reports, the ‘introduction’ and the ‘conclusion’ sections generate most, if not all, of the key phrases associated with that document. However, it is difficult to identify the structure in unstructured documents such as emails, blogs or meeting transcripts, which make them less amenable to easy detection of key phrases. Key phrases are typically closely related to the topic of the document. When there are multiple topics being discussed in the document, each topic contributes to key phrase candidate generation. This implies that using the structure of the document, such as the beginning and end of the document, as the major source of key phrases can result in missing key phrases arising from intermediate topic changes occurring in the document.

Another important factor that can be used in pruning key phrases in Step 3 is that key phrases are typically related to each other through a common topic. Here, again, this fact holds true for structured documents such as scientific articles. However, informal communications such as blogs, emails, etc, contain key phrases belonging to multiple topics and, hence, topic correlation is poor among key phrase candidates in such documents.

Step 2 of the key phrase extraction technique employs a learning algorithm to prune the candidates. Supervised learning algorithms for key phrase extraction consist of two major items, namely, task reformulation and feature identification. One approach is to formulate the problem as a binary classification, wherein the learning function is a binary classifier trained on both positive and negative key phrases. One of the disadvantages of using a binary classifier is that the algorithm does not rank the candidate key phrases. If there are key phrases that are more representative than others, the less relevant ones are not filtered by the binary classifier, which simply provides a ‘Yes’ or ‘No’ as an answer. This issue can be overcome by training a ranking classifier, which ranks the two key phrase candidates and ranks one more representative than the other.

Features that are used to represent a sample key phrase instance in supervised key phrase extraction typically include the length of the document, the document’s structure such as where in the document the phrase appears, ‘parts of speech’ tagging for the candidate phrase such as whether a word is a noun or verb, external features such as whether the candidate phrase appears in the title of a Wikipedia entry or in the query log of a search engine, etc.

Unsupervised approaches to key phrase candidate pruning include graph based approaches, wherein key phrase candidates are represented as nodes in a graph. An edge between two nodes connects two related

candidates. This idea is very similar to the popular page ranking algorithm adopted in search engines, where a node is considered important if it is related to a large number of other nodes and is connected to candidates that themselves are important. Each edge is treated as a vote for a candidate key phrase, and the edge weights are used to label the importance of the edges connecting the nodes. The score of a node is the summation of the edge weights, and the score is used to rank the candidate key phrases. One of the popular unsupervised automatic key phrase extraction techniques based on graph based ranking is the TextRank algorithm described in the paper <http://web.eecs.umich.edu/~mihalcea/papers/mihalcea.emnlp04.pdf>. A Java implementation of the same is available in GitHub at <https://github.com/ceter1/textrank>.

Another unsupervised learning method for key phrase extraction is using topic modelling of the document. Topics of the document are identified first and the key phrase candidates are grouped under the topics. Since key phrases are supposed to be representatives of the topics covered in the document, this can be modelled as a set-cover problem, where the key phrases are selected to cover all the topics of the document. Unlike the learning based approaches, language modelling based key phrase extraction techniques are becoming popular. We will discuss these in our next column.

Here is a takeaway exercise for our readers. We are all familiar with tag clouds which are built for Web pages. Given a Wikipedia article entry, can you come up with an algorithm to create a tag cloud for the same using any of the automatic key phrase extraction techniques we have discussed in this column? You can use any of the popular NLP toolkits such as NLTK for this exercise.

If you have any favourite programming questions or software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently in the happy land of ‘between jobs’. Her interests include compilers, storage systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya’s LinkedIn group Computer Science Interview Training India at <http://www.linkedin.com/groups?home=HYPERLINK%20http://www.linkedin.com/group%20s?home=&gid=2339182%20&HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20gid=2339182>